

Innlevering 2 – IN2010 - philipef

Deloppgave 1)

Jeg har implementert Insertion- og Quick-sorteringsalgoritmen. I tillegg valgte jeg Selection-sorteringsalgoritmen som har en kjøretid på $O(n^2)$. Til slutt valgte jeg Merge-sorteringsalgoritmen som har en kjøretid på $O(n \log(n))$.

For å teste om sorteringsalgoritmene ga rimelige svar, implementere jeg dem inn i Innlevering2Runner.java fra semestersiden. Jeg prøvde med forskjellige input-filer for å sjekke om de sorterte elementene i filene.

Deloppgave 2)

Jeg har målt bytter ved å gjøre det manuelt istedenfor å bruke metodene som var i Sorter.java. Grunnen til dette er at jeg ikke fikk ønsket output når jeg brukte den allerede eksisterende metoden. Jeg husket fortsatt å øke "swaps" variabelen, slik at resultatene ble rimelige.

Samtidig klarte jeg å få swap-metoden til å fungere i quickSort uten noen problemer.

For å måle sammenligningene brukte jeg hjelpemetodene i Sorter.java. Jeg brukte dem alle steder i sorteringsalgoritmene der variabler ble sammenlignet med hverandre. Det ga også et rimelig resultat.

Deloppgave 3)

Tester sorteringsalgoritmene for to filer: Fil_1 = "nearly_sorted_1000.txt" og Fil_2 = "random_1000.txt".

Måler tiden det tar for begge testene i Innlevering2Runner.java.

Selection-sort:

Tester for: "nearly_sorted_1000.txt":

Tid det tok i mikrosekunder: 423818

Antall bytter: 500499

Antall sammenligninger: 335334000

Tester for: "random_1000.txt":

Tid det tok i mikrosekunder: 419833
Antall bytter: 500499
Antall sammenligninger: 335334000

Insertion-sort:

Tester for: "nearly_sorted_1000.txt":

Tid det tok i mikrosekunder: 139241
Antall bytter: 241757
Antall sammenligninger: 1985014

Tester for: "random_1000.txt":

Tid det tok i mikrosekunder: 266340
Antall bytter: 81507743
Antall sammenligninger: 164513082

Quick-sort:

Tester for: "nearly_sorted_1000.txt":

Tid det tok i mikrosekunder: 479697
Antall bytter: 112851002
Antall sammenligninger: 226513883

Tester for: "random_1000.txt":

Tid det tok i mikrosekunder: 162796
Antall bytter: 2866355
Antall sammenligninger: 11053810

Merge-sort:

Tester for: "nearly_sorted_1000.txt":

Tid det tok i mikrosekunder: 172044
Antall bytter: 0
Antall sammenligninger: 20359841

Tester for: "random_1000.txt":

Tid det tok i mikrosekunder: 179508

Antall bytter: 0

Antall sammenligninger: 23540407

1. *I hvilken grad stemmer kjøretiden overens med kjøretidsanalysene (store O) for de ulike algoritmene?*

Svar:

Det stemmer overens med kjøretidsanalysen. Siden Insertionsort, Quicksort og Selectionsort har kjøretiden $O(n^2)$.

Insertionsort var spesielt rask til tross for kjøretidsanalysen. Samtidig er input-filen som ble brukt relativt liten, derfor er det ikke overraskende at det gikk så raskt, siden den er nevnt som den raskeste sorteringsalgoritmen for små arrayer. Den er spesielt rask for nesten sorterte arrayer som man kan se over. Den var nesten dobbelt så rask på "nearly_sorted_1000.txt" sammenlignet med "random_1000.txt".

Quicksort var rask på "random_1000.txt", men ikke på "nearly_sorted_1000.txt". Dette er muligens på grunn av at pivot-elementet var det midterste eller nærme det midterste elementet i arrayet. Hvis den var det vil kjøretiden være på $O(n \log(n))$.

Mergesort var nærmest like rask på begge filene, dette er mest sannsynlig på grunn av at den har en kjøretid på $O(n \log(n))$.

Man kan se at visse variabler spiller inn på resultatene, som at filen som ble valgt ikke var spesielt stor og at pivot-elementet i quicksort kan ha vært det midterste elementet. Jeg konkluderer med at kjøretiden stemmer i stor grad overens med de ulike algoritmene sin kjøretidsanalyse.

2. *Hvordan er antall sammenligninger og antall bytter korrelert med kjøretiden?*

Svar:

Mergesort var den sorteringsalgoritmen med konsistent lav kjøretid på begge filene. En kan se at antall bytter var 0 på begge

Insertionsort hadde svært få bytter og sammenligninger på "nearly_sorted_1000.txt", noe som viser til hvorfor det var den sorteringen som tok kortest tid.

I quicksort tilfellet kan en se at testen for "random_1000.txt" hadde færre bytter og sammenligninger enn testen for "nearly_sorted_1000.txt" og at det dermed tok kortere tid.

Det som gir mest grunnlag for å komme til en konklusjon er selectionsort testene. Grunnen til dette er at de hadde like mange bytter og sammenligninger, dermed ble tiden på begge testene svært like.

Kjøretiden avhenger ikke kun av den teoretiske kjøretidsanalysen og input-filene, men også av hvordan jeg har programmert algoritmene. Altså kan jeg ha programmert noen algoritmer mer eller mindre effektive enn andre som vil ha en effekt på slutt resultatene.

På grunn av observasjonene kan man konkludere med at antall sammenligninger og antall bytter har en effekt på kjøretiden, da kjøretiden øker med et økt antall bytter og/eller sammenligninger.

3. Hvilke sorteringsalgoritmer utmerker seg positivt når n er veldig liten? Og når n er veldig stor?

Svar:

Tester for filene "nearly_sorted_10.txt" og "random_10.txt".

Selection-sort:

Tester for "nearly_sorted_10.txt":

Tid det tok i mikrosekunder: 44904

Antall bytter: 54

Antall sammenligninger: 540

Tester for "random_10.txt":

Tid det tok i mikrosekunder: 42616

Antall bytter: 54

Antall sammenligninger: 540

Insertion-sort:

Tester for "nearly_sorted_10.txt":

Tid det tok i mikrosekunder: 44476

Antall bytter: 21

Antall sammenligninger: 216

Tester for "random_10.txt":

Tid det tok i mikrosekunder: 41724

Antall bytter: 75

Antall sammenligninger: 317

Quick-sort:

Tester for "nearly_sorted_10.txt":

Tid det tok i mikrosekunder: 40484

Antall bytter: 159

Antall sammenligninger: 436

Tester for "random_10.txt":

Tid det tok i mikrosekunder: 42066

Antall bytter: 120

Antall sammenligninger: 376

Merge-sort:

Tester for "nearly_sorted_10.txt":

Tid det tok i mikrosekunder: 41420

Antall bytter: 0

Antall sammenligninger: 1084

Tester for "random_10.txt":

Tid det tok i mikrosekunder: 42716

Antall bytter: 0

Antall sammenligninger: 1127

Ut ifra observasjonene over, ser alle relativt likeverdige når n er svært liten. Siden n er så liten vil tidsforskjellene bety mye. Quicksort er den som utmerker seg positivt siden den har den laveste målte tiden for "nearly_sorted_10.txt" og den andre beste tiden for "random_10.txt".

Tester for filene "nearly_sorted_10000.txt" og "random_10000.txt".

Selection-sort:

Tester for "nearly_sorted_10000.txt":

Tid det tok i mikrosekunder: 284950281
Antall bytter: 50004999
Antall sammenligninger: 1474109088

Tester for "random_10000.txt":

Tid det tok i mikrosekunder: 289068540
Antall bytter: 50004999
Antall sammenligninger: 1474109088

Insertion-sort:

Tester for "nearly_sorted_10000.txt":

Tid det tok i mikrosekunder: 662305
Antall bytter: 22170525
Antall sammenligninger: 194346050

Tester for "random_10000.txt":

Tid det tok i mikrosekunder: 127702627
Antall bytter: 1382021123
Antall sammenligninger: 1381008387

Quick-sort:

Tester for "nearly_sorted_10000.txt":

Tid det tok i mikrosekunder: NULL
Antall bytter: NULL
Antall sammenligninger: NULL

Tester for "random_10000.txt":

Tid det tok i mikrosekunder: 3957879
Antall bytter: 396587073
Antall sammenligninger: 1559695665

Merge-sort:

Tester for "nearly_sorted_10000.txt":

Tid det tok i mikrosekunder: 3628295

Antall bytter: 0

Antall sammenligninger: 1686315573

Tester for "random_10000.txt":

Tid det tok i mikrosekunder: 6090514

Antall bytter: 0

Antall sammenligninger: 1200332867

Når n er stor er Insertion-sort på arrayer som er nesten sortert svært effektiv.

På grunn av at Mergesort har $O(n \log(n))$, så er den svært effektiv på begge filene.

4. Hvilke sorteringsalgoritmer utmerker seg positivt for de ulike inputfilene?

Svar:

Selectionsort utmerker seg ikke spesielt positivt for noen av inputfilene, men den holder en stabil kjøretid for alle.

Insertionsort utmerker seg positivt for alle nesten sorterte filer.

Quicksort utmerker seg positivt for alle inputfiler mindre enn 10 000 elementer.

Mergesort utmerker seg positivt for alle inputfilene, men spesielt godt for inputfiler der n er stor.

5. Har du noen overraskende funn å rapportere?

Svar:

Et overraskende funn jeg fant var at Quick sorteringsalgoritmen får en stackoverflow error, når inputfilen er lik 10000 elementer eller mer på sorterte arrayer.

Et annet overraskende funn er at Selection sorteringsalgoritmen gir opp nær slutten av input-filer med 10000 elementer eller mer visse ganger, men kun for nærmest sorterte arrayer.